

Imbalance at Risk (IaR) — MVP / PoC 4-Week Project Plan

Team: 2 data scientists (DS1, DS2), ~full-time **Goal:** A local, Streamlit-based PoC that ingests Optimeering price forecasts + flat-file portfolio data, runs a Monte Carlo IaR simulation under the MVP's independence assumption, stores results in SQLite, surfaces a dashboard with limits/alerts, and backtests past IaR estimates against realised imbalance cost. **Scope guardrail:** DAM-only, 2–3 users, single price area per portfolio, independence between price and imbalance assumed. Correlation modelling, intraday/balancing positions, multi-area netting, live trading, cloud deploy = out of scope.

Guiding sequencing principles

- **Week 1 = data flowing end-to-end.** Optimeering API → SQLite → a basic imbalance number, even on stub portfolio data, so the riskiest external assumption (the API) is validated on day one.
- **Backtesting lands by Week 3** — it is a core MVP requirement and needs both actual delivery (flat file) and actual historical prices (Optimeering), so it cannot be left to the final week.
- **Week 4 = harden, document, demo.** No new core features.
- A thin **calculation backend separated from the Streamlit UI** is established in Week 1 so the two people can work in parallel without colliding in the UI layer.

Weekly milestones at a glance

Week	Milestone	Headline deliverable
1	Data plumbing proven	Optimeering price forecast + flat-file positions land in SQLite; trivial imbalance computed end-to-end
2	Simulation engine working	Monte Carlo IaR (Gross + Spread) over a 24h horizon, persisted to DB, validated on synthetic cases
3	Backtesting + alerts	Historical performance module (exceedance frequency / Kupiec), configurable limits, alert flags
4	Dashboard + demo	Streamlit dashboard matching mockups, tests, docs, end-to-end demo

Detailed task table

Week 1 — Get data flowing end-to-end (validate assumptions early)

Week	Task	Description	Owner	Depends On	Deliverable
1	1.1 Repo + project skeleton	Create repo, package layout (<code>ingestion/</code> , <code>db/</code> , <code>simulation/</code> , <code>backtest/</code> , <code>app/</code>), virtualenv, dependency pinning, README stub. Establish the backend-vs-UI separation up front.	DS1	—	Runnable empty package, <code>pip install -e .</code> works
1	1.2 SQLite schema v1	Define core tables: User, Portfolio, DAMPosition, GenerationForecast, ActualDelivery, ImbalancePriceForecast, ActualImbalancePrice, SimulationRun, IaRResult, Alert, HistoricalPerformanceRecord. Use SQLAlchemy models + a migration/init script.	DS2	1.1	<code>schema.sql</code> / SQLAlchemy models + <code>init_db()</code>
1	1.3 Optimeering API client	Thin client wrapping (a) forward imbalance price predictions (quantiles/distribution) and (b) historical actual prices, by price area + horizon. API-key auth, retry/backoff, error handling, local response caching to avoid redundant calls.	DS1	1.1	<code>optimeering_client.py</code> with two methods + cached responses
1	1.4 Flat-file loader	CSV/Excel loaders for DAM positions, generation forecasts, actual delivery; validate schema, parse timestamps to MTU index, write to SQLite.	DS2	1.2	<code>flatfile_loader.py</code> , sample files load cleanly
1	1.5 End-to-end smoke path	Stub portfolio (1 user, 1 area, e.g. NO2). Pull a real price forecast via 1.3, load stub positions via 1.4, compute trivial expected imbalance = DAM – forecast gen, persist. Confirms the full chain works.	DS1 + DS2	1.3, 1.4	Script that runs API → DB → imbalance number; screenshot/log
1	1.6 Optimeering data contract note	Document the actual API response shape, statistic types (which quantiles exist), units, area codes, time alignment, gaps. This de-risks Week 2.	DS1	1.3	1-page data contract / field map

Week 1 exit criteria: real Optimeering data and stub positions both sit in SQLite; a single imbalance figure is computed and stored; team knows exactly what the price-forecast distribution looks like.

Week 2 — Monte Carlo simulation engine

Week	Task	Description	Owner	Depends On	Deliverable
2	2.1 Imbalance uncertainty model	Turn expected imbalance (DAM – forecast gen) into a <i>distribution</i> per MTU. For MVP with no historical error data yet: parametric forecast-error model (e.g. Normal or scaled-t around the point forecast, sigma from a configurable % of capacity or forecast). Document the assumption explicitly.	DS2	1.5, 1.6	<code>imbalance_model.py</code> returning per-MTU distribution params

Week	Task	Description	Owner	Depends On	Deliverable
2	2.2 Price marginal sampler	Convert Optimeering quantile forecasts into a sampleable marginal per MTU (interpolate the quantile function / inverse-CDF sampling). Preserve the asymmetry/heavy tail present in the forecast rather than refitting to Normal.	DS1	1.6	<code>price_sampler.py</code> , draws match input quantiles
2	2.3 Monte Carlo core	Sample independently from (a) price marginal and (b) imbalance distribution per MTU, $\geq 5,000$ scenarios; compute per-scenario P&L; aggregate to Gross IaR (position $\times$ price) and Spread IaR (position $\times$ (imbalance price – DAM)). Period IaR = quantile of summed P&L across MTUs (joint quantile, not summed per-MTU IaRs). Also output ClaR/Expected Shortfall.	DS1	2.1, 2.2	<code>engine.py</code> , returns IaR + ClaR by confidence + horizon
2	2.4 Persist simulation runs	Write SimulationRun (params, seed, timestamp, config) + IaRResult rows (by confidence level and horizon) to SQLite; design for efficient later querying by portfolio + vintage.	DS2	2.3, 1.2	Runs + results stored and re-queryable
2	2.5 Engine validation	Sanity tests: known analytic cases (e.g. all-Normal collapses to closed-form VaR-like number), convergence of P05 vs scenario count, reproducibility under seed. Catch the "summing per-MTU IaR" trap explicitly.	DS2	2.3	Test notebook/script + short validation note

Week 2 exit criteria: a stored IaR number (Gross + Spread, with ClaR) for the stub portfolio over a 24h horizon, computed from real price forecasts and a documented imbalance-uncertainty model, with basic validation passing.

Week 3 — Backtesting, limits, alerts (core MVP, scheduled no later than now)

Week	Task	Description	Owner	Depends On	Deliverable
3	3.1 Realised cost computation	Ingest actual delivery (flat file → DB), fetch actual historical imbalance prices (Optimeering), compute actual imbalance = DAM – actual delivery, then realised imbalance cost = actual imbalance $\times$ actual price, per MTU/hour.	DS2	1.3, 1.4	<code>realised_cost.py</code> , historical cost series in DB
3	3.2 IaR vintage replay	Generate (or backfill) historical IaR estimates over a past period so each period has a stored "estimate-as-of" vintage to compare against. Define the comparison join: IaR estimate vintage vs the realised cost that actually occurred.	DS1	2.3, 2.4	Backfilled IaRResult vintages over a test window
3	3.3 Exceedance / calibration test	Compare realised cost vs stored IaR estimate; compute exceedance frequency; check $\approx 5\%$ breaches at P95. Implement a Kupiec POF test for calibration. Store HistoricalPerformanceRecord rows.	DS1	3.1, 3.2	<code>backtest.py</code> + Kupiec result + exceedance stats
3	3.4 Limits & alerts module	Configurable per-portfolio €-limits (per-MTU, rolling window, remaining-day per the deck's horizons). Evaluate latest IaR against limit, flag breaches, write Alert rows with severity.	DS2	2.4, 1.2	<code>alerts.py</code> , limit config, breach flags persisted

Week	Task	Description	Owner	Depends On	Deliverable
3	3.5 Backend API surface	Freeze a clean internal function/service interface the UI will call (get_latest_iar, get_iar_curve, get_alerts, get_backtest_summary). Keeps Streamlit thin.	DS1 + DS2	2.3, 3.3, 3.4	Documented backend interface module

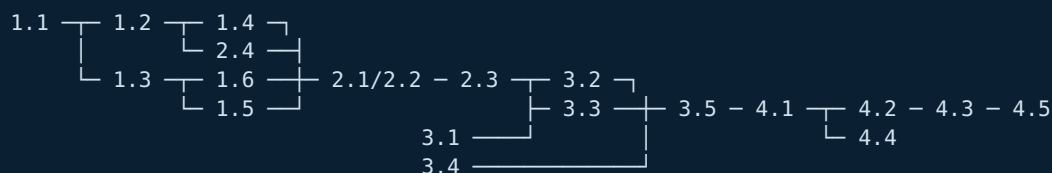
Week 3 exit criteria: the key MVP metric works — past IaR estimates compared to realised cost with an exceedance/Kupiec readout — plus configurable limits producing alert flags. All consumable through a stable backend interface.

Week 4 — Dashboard, testing, documentation, demo

Week	Task	Description	Owner	Depends On	Deliverable
4	4.1 Streamlit dashboard	Build UI per the mockups: IaR curve(s) over horizon with limit line + alert status, colour-coded risk heatmap by MTU/hour, Gross/Spread toggle, confidence selector, portfolio/area picker. Reads only via the backend interface (3.5).	DS1	3.5	Running Streamlit app
4	4.2 Backtest view	Dashboard panel showing exceedance frequency, the ~5% target line, and Kupiec pass/fail — the calibration story for the demo.	DS2	3.3, 4.1	Backtesting dashboard panel
4	4.3 End-to-end test pass	Integration test of the full chain (ingest → simulate → store → alert → display) on 2–3 real-ish portfolios in different areas (NO1/NO2/SE3); fix breakages.	DS2	4.1, 4.2	Green end-to-end run on multiple portfolios
4	4.4 Documentation	README + run instructions, data-contract doc, explicit assumptions log (independence, imbalance-uncertainty model, single-price settlement) and known limitations vs the full IaR vision.	DS1	4.1	docs/ with setup + assumptions + limitations
4	4.5 Demo prep + dry run	Demo script and dataset, walk the narrative from the deck (position report → probabilistic IaR → limits/alerts → backtest), rehearse, capture screenshots.	DS1 + DS2	4.1, 4.2, 4.3	Demo deck/script + rehearsed run

Week 4 exit criteria: a demoable dashboard on multiple portfolios, calibration story visible, documented assumptions, rehearsed demo.

Critical-path dependency summary



The longest chain runs through the **simulation engine (2.3)** and then **backtesting (3.3)** into the **backend interface (3.5)** and **UI (4.1)**. Anything that slips 2.3 slips the whole back half, which is why Week 1 de-risks the API and data shape before the engine is built.

Top technical risks and mitigations

Risk 1 — Optimeering API: shape, quantile availability, latency, or access surprises. The entire simulation depends on the price-forecast distribution and on historical actuals coming back in a usable, well-understood form. If the quantiles are coarse, the area codes don't map, timestamps don't align to MTUs, or rate limits bite, Week 2 stalls. *Mitigation:* tackle it in Week 1 (tasks 1.3, 1.6), wrap in a thin client with **caching** so the team isn't hammering the API or blocked by transient failures, and write the data-contract note before building the sampler. Build the price sampler (2.2) to consume whatever quantile granularity actually exists rather than an assumed one.

Risk 2 — Imbalance-uncertainty model is under-determined for the MVP. The brief requires modelling the uncertainty of (DAM – forecast generation), but the team has no historical forecast-error distribution yet. A poorly chosen assumption makes IaR meaningless and the backtest will expose it. *Mitigation:* use an explicit, documented parametric forecast-error model (sigma as a configurable fraction of forecast/capacity), keep it a single swappable module (2.1), and treat the **Week 3 backtest as the validation loop** — if exceedance frequency is far from ~5%, recalibrate sigma. Log the assumption prominently so the demo audience knows it's a known simplification.

Risk 3 — The MVP's independence assumption understates true IaR (per the deck's Challenges 1–3). The background deck makes a strong point that ignoring price → position dependence and fat tails can understate IaR by ~70%. The MVP deliberately assumes independence — correct for scope, but the numbers will look optimistic, and the backtest may show too few exceedances. *Mitigation:* this is sanctioned scope, not a bug — so manage it as a **communication risk**. Preserve heavy tails on the price side by sampling from Optimeering's actual quantiles (don't refit to Normal), document independence in the assumptions log, and frame it in the demo as the explicit next-step on the roadmap toward "true IaR" (copula-based dependence). The backtest panel makes this honest and visible rather than hidden.

Notes on alignment with the background materials

- **Gross vs Spread IaR** (deck p.11) are both produced by the engine (task 2.3).
- **ClaR / Expected Shortfall** (deck p.12) is included as an engine output.
- **Horizons** (deck p.13): per-MTU, rolling window, remaining-day map onto the limit types in task 3.4.
- **Joint period quantile, not summed per-MTU IaRs** (deck p.16) is explicitly handled in 2.3 and tested in 2.5.

- **Kupiec calibration test** (deck p.20) is the backtest metric in task 3.3.
- **Dashboard** (deck p.18): IaR vs limit, colour-coded MTU heatmap, alerts-before-breach drive the UI in task 4.1.
- **Deliberately out of MVP scope but central to the full vision:** fat-tail-aware *modelling* beyond the supplied quantiles, price ↔ position dependence (copula), and cross-MTU autocorrelation. Flagged as roadmap, per Risk 3.